

# BMI Calculator — Task 2

Viva Preparation Notes | Internship Task Series

## 1. Project Overview

Ye task ek BMI (Body Mass Index) Calculator hai jo do tiers mein banaya gaya: Beginner tier ek command-line tool hai, aur Advanced tier ek full GUI application hai jisme database aur graph bhi shamil hain. Objective ye hai ke user ka weight aur height le kar BMI calculate kiya jaaye aur usko health category mein classify kiya jaaye.

- **Beginner:** weight (kg) aur height (m) input, BMI = weight / height<sup>2</sup>
- **Advanced:** tkinter GUI + SQLite database + matplotlib trend graph + multi-user support

## 2. Beginner Tier — Key Concepts

### 2.1 Input aur Validation

Python ka input() function hamesha string return karta hai, isliye pehle float() se convert karna padta hai. Agar user letters ya symbols daale to ValueError aata hai — isko try/except mein pakadna hain taake program crash na ho, bas dobara pooch le. Negative ya zero value bhi reject hoti hai chahe number valid ho (separate if check).

### 2.2 BMI Formula

BMI = weight (kg) ÷ height<sup>2</sup> (m<sup>2</sup>). Height<sup>2</sup> Python mein height \*\* 2 se likha jaata hai. Result ko :.2f format specifier se 2 decimal places tak round kiya jaata hai.

### 2.3 Classification (if-elif chain)

- **Underweight** — BMI < 18.5
- **Normal weight** — 18.5 ≤ BMI < 25
- **Overweight** — 25 ≤ BMI < 30
- **Obese** — BMI ≥ 30

Har condition upar se check hoti hai isliye sirf < likhna kaafi hai, jaise 25 se kam hi 'Normal' category mein reh jaata hai (Underweight already upar filter ho chuka).

### 2.4 Loop for Repeat Use

Pura program while True loop ke andar hai taake user restart kiye bagair multiple baar calculate kar sake — 'n' type karne par hi loop break hota hai.

## 3. Advanced Tier — Key Concepts

### 3.1 GUI aur Event-Driven Programming

tkinter ek event-driven library hai — CLI ki tarah top-to-bottom nahi chalti, balke widgets bana kar app.mainloop() call karte hain jo user ke clicks/keypresses ka wait karta hai aur har button ka command callback function trigger karta hai (jaise Calculate button ka command=self.on\_calculate).

### 3.2 OOP Structure

BMIApp class tk.Tk se inherit karti hai (class BMIApp(tk.Tk):), matlab BMIApp khud ek window hai. \_\_init\_\_ mein super().\_\_init\_\_() call karke parent class ka setup complete karte hain, phir widgets banate hain.

### 3.3 Code Reuse (DRY Principle)

calculate\_bmi() aur classify\_bmi() functions beginner file se hi import kiye gaye hain (from bmi\_calculator\_beginner import ...) — same logic dobara likhne ki zaroorat nahi. Ye Don't Repeat Yourself (DRY) principle ka example hai.

### 3.4 Modular Design

Database ka pura kaam alag file bmi\_db\_utils.py mein hai, GUI se separate. Ye separation of concerns hai — agar kal database SQLite se kisi aur system mein badalna ho, to sirf ek file change karni padegi, GUI code touch nahi karna padega.

### 3.5 SQLite Database

- CREATE TABLE IF NOT EXISTS — table sirf pehli baar banti hai, dobara run karne par error nahi aata.

- Parameterized queries (? placeholders) — values ko query string mein directly join nahi karte, isse SQL Injection attack se bachav hota hai.
- with `sqlite3.connect(...)` as `conn` — context manager hai, connection apne aap close/commit ho jaata hai, chahe error aaye ya na aaye.

### 3.6 Error Handling Pattern

Har database function exception ko raise nahi karta — balke (True, data) ya (False, error\_message) tuple return karta hai. GUI ye check karke user ko `messagebox.showerror()` se friendly error dikhati hai, app crash nahi hoti.

### 3.7 matplotlib Embedding

Normal `matplotlib.pyplot.show()` apni alag window kholta hai. Yahan Figure object aur `FigureCanvasTkAgg` use kiya gaya hai taake chart seedha tkinter Toplevel window ke andar render ho — GUI ka hi hissa lage, separate popup na ho.

### 3.8 Case-Insensitive Multi-User Support

Name field se pata chalta hai record kis user ka hai. 'dua', 'DUA', aur 'Dua' ko same user maanne ke liye `normalize_username()` function name ko `.strip().title()` se normalize karta hai (hamesha 'Dua' banega), aur DB query mein extra safety ke liye `COLLATE NOCASE` bhi laga hai.

## 4. Likely Viva Questions & Answers

---

**Q: BMI calculate karne ka formula kya hai?**

**A:** BMI = weight (kg) / height (m)<sup>2</sup>. Height ko meters mein hona chahiye, kg/m<sup>2</sup> unit hoti hai.

**Q: input() se li gayi value seedhi arithmetic mein use kyun nahi kar sakte?**

**A:** Kyunke `input()` hamesha string return karta hai, chahe user number hi type kare. Isliye `float()` se explicitly convert karna zaroori hai.

**Q: try/except ka is program mein kya role hai?**

**A:** `float()` conversion fail ho (jaise letters type ho) to `ValueError` aata hai — `try/except` isko catch karke crash hone se bachata hai aur user ko dobara input lene deta hai.

**Q: Command-line tool aur GUI application mein basic difference kya hai?**

**A:** CLI top-to-bottom sequentially chalta hai aur `input()` se ruk kar wait karta hai. GUI event-driven hoti hai — `mainloop()` chalta rehta hai aur sirf user ke action (click, type) par respective function trigger hota hai.

**Q: SQLite mein parameterized queries (?) kyun use karte hain, seedha string format kyun nahi?**

**A:** Taake SQL Injection se bacha ja sake — agar user input query string mein directly concatenate karein to malicious input query ka structure badal sakta hai. ? placeholders values ko safely escape karte hain.

**Q: Application mein multi-user support kaise implement hua hai?**

**A:** Har record ke saath username DB mein save hota hai. History aur graph fetch karte waqt sirf current entered username ke records query kiye jaate hain (`WHERE username = ?`).

**Q: Case-sensitivity ka issue kaise solve kiya gaya?**

**A:** `normalize_username()` function har naam ko `strip()` aur `title()` se ek consistent format mein convert karta hai, isliye 'dua' aur 'DUA' dono 'Dua' ban kar same user ke record maane jaate hain.

**Q: Advanced tier mein database function GUI ko crash kyun nahi karte?**

**A:** Har DB function apna kaam `try/except` mein karta hai aur (success, result) ya (False, error\_message) tuple return karta hai — exception function ke andar hi handle ho jaata hai, GUI ko sirf result dikhana padta hai.

**Q: FigureCanvasTkAgg kis liye use hota hai?**

**A:** Ye matplotlib ke chart ko tkinter window ke andar embed karne deta hai, taake graph app ka hi part lage — alag se koi matplotlib popup window na khule.

**Q: Beginner aur Advanced tier mein code duplicate kyun nahi kiya gaya?**

**A:** DRY (Don't Repeat Yourself) principle follow kiya gaya — advanced file beginner file se `calculate_bmi()` aur `classify_bmi()` functions import karti hai, taake logic ek hi jagah maintain ho.

**Q: Database file (bmi\_records.db) kab banti hai?**

**A:** `init_db()` function pehli baar app run hone par `CREATE TABLE IF NOT EXISTS` query chalata hai, jisse file aur table dono automatically ban jaate hain agar pehle se exist nahi karte.

**Q: Result ka colour-coding kaise decide hota hai?**

**A:** Ek dictionary (`CATEGORY_COLORS`) hai jisme har category (Underweight, Normal, Overweight, Obese) ka ek hex colour code map kiya gaya hai — result label ka fg (text colour) usi dictionary se set hota hai.

## 5. Quick Glossary

---

- **Event-driven programming** — Ek function/library ka behaviour jo user ke actions (click, keypress) ka wait kare aur usi par react kare — CLI ke seedhe top-to-bottom flow ke ulat.
- **Context manager** — Ek code block jo kisi cheez ko setup aur automatically cleanup/close karta hai (with statement) — connection close karna bhool jaane ka risk nahi rehta.
- **Parameterized query** — Query mein ? ya %s jaisi jagah chhod kar values ko baad mein safely pass karna — SQL Injection se bachaata hai.
- **Modular design** — Code ko chhoti, alag-alag files/functions mein todna taake har hisse ki responsibility clear ho (jaise db\_utils.py alag from GUI).
- **DRY Principle** — Same logic ko dobara na likh kar ek jagah rakhna aur wahin se reuse/import karna.
- **Widget** — GUI ke chhote elements — Label, Entry (text box), Button, Treeview (table) waghera.

### □ PROJECT KA MUKHTASAR JA'IZA (Overview)

Hum ne aik **2-Tier** application banai:

1. **Beginner Tier (CLI)**: Command line par chalne wali, jis mein input validation aur basic math thi.
2. **Advanced Tier (GUI)**: Tkinter window wali, jis mein database (SQLite), multi-user support, aur matplotlib graph tha.

Hum ne 3 Python files banayin:

- bmi\_calculator\_beginner.py (Math logic + CLI)
  - bmi\_db\_utils.py (Database helper functions)
  - bmi\_calculator\_advanced.py (GUI application)
- 

### □ 1. PYTHON CORE CONCEPTS (Beginner Tier)

- `while True` (**Infinite Loop**): Tab tak chalna jab tak user sahi input na de.
  - `continue`: Loop ke current round ko wahan rok kar agle round par jump kar jana (hamne isay error ke baad dobara input lene ke liye use kiya).
  - `try/except ValueError`: Jab user number ki jagah "abc" type kare, toh program crash nahi hoti, balke hum error handle kar ke dobara poochte hain.
  - `.strip()`: String ke aage/peeche ki spaces hatao (jaise " dua " -> "dua").
  - `.title()`: Har word ka pehla letter capital (bara) aur baqi small (chhota) kar do (jaise "dua" -> "Dua").
  - `:.2f`: Float number ko sirf 2 decimal places tak dikhana (jaise 22.493 -> 22.49).
  - `if __name__ == "__main__":`: Yeh file direct chale toh `main()` chalao, agar import ho toh nahi chalao. Is se code reusable aur clean rehta hai.
- 

### □ 2. TKINTER (GUI Development)

- `tk.Tk()`: Main window banai.
- `Entry`: Text box jahan user data type karta hai (name, weight, height).

- `Button`: Click karne par `command` wala function chalta hai (jaise `on_calculate`)-
  - `grid()` / `pack()`: Widgets ko window mein arrange (set) karne ke tareeqay.
  - `Label`: Text display karna (jaise result ya heading).
  - `ttk.Treeview`: Modern table (spreadsheet) jahan history display hoti hai (columns: Date, Weight, BMI, etc.).
  - `messagebox`: Pop-up alerts ke liye (error, info, warning)-
  - `Toplevel()`: Main window ke andar se **nayi window** kholna (jaise graph wali window)-
- 

### □ 3. SQLITE (Database)

- `sqlite3.connect()`: Database file se connection kholna.
  - `with ... as conn::` Context Manager. Is se `commit()` (save), `rollback()` (undo), aur `close()` **khud ba khud** ho jata hai. Agar error aaye toh changes wapas (rollback) ho jate hain.
  - **Parameterized Queries (?)**: SQL Injection attacks se bachne ke liye, values ko seedha query mein mix nahi karte, balke `(?, ?, ...)` use karte hain.
  - `fetchall()`: SELECT query ke **saare results** ko list mein laana (chahe 0 rows hon ya 1000)-
  - `fetchone()`: 1<sup>st</sup> column return only
  - `row[0]` / **List Comprehension** `[row[0] for row in rows]`: Database se aaye tuples ki list mein se sirf pehli value (username) nikaal kar simple list banana.
  - `COLLATE NOCASE`: SQLite mein sorting karte waqt chote/bade letters (case) ko ignore karna (taake `Dua` aur `dua` ek saath aayein)-
- 

### □ 4. MATPLOTLIB (Graphs)

- `Figure(figsize, dpi)`: Drawing board (canvas) banai. `figsize` inch mein size hai, `dpi` (Dots Per Inch) quality control karta hai. Zyada `dpi` = Clear (sharp), kam `dpi` = Blurry.
  - `add_subplot(111)`: Graph ka area banaya (1 row, 1 column, 1st plot)-
  - `plot.plot(dates, bmis, marker="o")`: Data points ko line aur dots ke saath draw karna.
  - `tick_params(rotation=45)`: X-axis (dates) ki text ko 45 degree ghumana taake overlap na ho aur parhne mein aasan ho.
  - `FigureCanvasTkAgg`: Matplotlib ke `Figure` ko **tkinter widget** mein convert (translate) karna taake woh GUI window mein dikhe.
  - `canvas.draw()`: Graph ko render (dikhaana)-
- 

### □ 5. GIT & GITHUB (Version Control)

- `git init`: Folder ko Git repository mein convert karna.
- `git add .`: Saari changes (modified files) ko staging area mein daalna (commit ke liye ready karna)-
- `git commit -m "message"`: Staged files ko local history mein save karna.
- `git push`: Local commits ko **GitHub (Remote)** par upload karna.
- `.gitignore`: Un files ko ignore karna jo upload nahi karni (jaise `.mp4` videos, `__pycache__` folders) taake repo clean rahe.
- `git rm --cached`: File ko GitHub tracking se hatana lekin local folder mein rakna.

---

## □ 6. CODE STRUCTURE & ARCHITECTURE

- **Tuple Unpacking:** `name, weight, height = parsed` (Ek dabbe/tuple ki values ko alag alag variables mein daalna)-
  - **Importing Modules:** `from file import function` ya `import file as alias` (jaise `import bmi_db_utils as db`)-
  - **Classes (OOP):** GUI ko `class BMIApp(tk.Tk) :` ke andar wrap kiya. `__init__` constructor hai jo window set karta hai.
  - **Separation of Concerns:**
    - DB ka kaam alag (`db_utils`)-
    - Math ka kaam alag (`beginner`)-
    - GUI ka kaam alag (`advanced`)- Is se code parhna aur debug karna aasan ho jata hai.
- 

## ✓ FINAL DELIVERABLE (Aap ne kya achieve kiya?)

- **CLI Version:** Input validation ke saath BMI calculator.
- **GUI Version:** Colour-coded results, multi-user support, SQLite persistence, aur interactive history table.
- **Graph Window:** Matplotlib ka line chart jo BMI trend ko date-wise dikhata hai.
- **GitHub Repository:** `OIBSIP` naam ka public repo jahan saara code push hai (README ke saath)-